SQL Queries Interview Questions - Oracle Part 1

As a database developer, writing SQL queries, PLSQL code is part of daily life. Having a good knowledge on SQL is really important. Here i am posting some practical examples on SQL queries.

To solve these interview questions on SQL queries you have to create the products, sales tables in your oracle database. The "Create Table", "Insert" statements are provided below.

```
CREATE TABLE PRODUCTS

(

       PRODUCT_ID     INTEGER,

       PRODUCT_NAME   VARCHAR2(30)

);

CREATE TABLE SALES

(

       SALE_ID        INTEGER,

       PRODUCT_ID     INTEGER,

       YEAR           INTEGER,

       Quantity       INTEGER,

       PRICE          INTEGER

);



INSERT INTO PRODUCTS VALUES ( 100, 'Nokia');

INSERT INTO PRODUCTS VALUES ( 200, 'IPhone');

INSERT INTO PRODUCTS VALUES ( 300, 'Samsung');

INSERT INTO PRODUCTS VALUES ( 400, 'LG');



INSERT INTO SALES VALUES ( 1, 100, 2010, 25, 5000);
```

```
INSERT INTO SALES VALUES ( 2, 100, 2011, 16, 5000);

INSERT INTO SALES VALUES ( 3, 100, 2012, 8,  5000);

INSERT INTO SALES VALUES ( 4, 200, 2010, 10, 9000);

INSERT INTO SALES VALUES ( 5, 200, 2011, 15, 9000);

INSERT INTO SALES VALUES ( 6, 200, 2012, 20, 9000);

INSERT INTO SALES VALUES ( 7, 300, 2010, 20, 7000);

INSERT INTO SALES VALUES ( 8, 300, 2011, 18, 7000);

INSERT INTO SALES VALUES ( 9, 300, 2012, 20, 7000);

COMMIT;
```

The products table contains the below data.

```
SELECT * FROM PRODUCTS;



PRODUCT_ID PRODUCT_NAME

-----------------------

100        Nokia

200        IPhone

300        Samsung
```

The sales table contains the following data.

```
SELECT * FROM SALES;
```

```
SALE_ID PRODUCT_ID YEAR QUANTITY PRICE

-------------------------------------

1       100        2010   25     5000

2       100        2011   16     5000

3       100        2012   8      5000

4       200        2010   10     9000

5       200        2011   15     9000

6       200        2012   20     9000

7       300        2010   20     7000

8       300        2011   18     7000

9       300        2012   20     7000
```

Here Quantity is the number of products sold in each year. Price is the sale price of each product.

I hope you have created the tables in your oracle database. Now try to solve the below SQL queries.

1. Write a SQL query to find the products which have continuous increase in sales every year?

Solution:

Here “Iphone” is the only product whose sales are increasing every year.

STEP1: First we will get the previous year sales for each product. The SQL query to do this is

```
SELECT P.PRODUCT_NAME,

       S.YEAR,

       S.QUANTITY,

       LEAD(S.QUANTITY,1,0) OVER (

                            PARTITION BY P.PRODUCT_ID

                            ORDER BY S.YEAR DESC
```

```
                            ) QUAN_PREV_YEAR

FROM   PRODUCTS P,

       SALES S

WHERE  P.PRODUCT_ID = S.PRODUCT_ID;



PRODUCT_NAME YEAR QUANTITY QUAN_PREV_YEAR

----------------------------------------

Nokia        2012    8         16

Nokia        2011    16        25

Nokia        2010    25        0

IPhone       2012    20        15

IPhone       2011    15        10

IPhone       2010    10        0

Samsung      2012    20        18

Samsung      2011    18        20

Samsung      2010    20        0
```

Here the lead analytic function will get the quantity of a product in its previous year.

STEP2: We will find the difference between the quantities of a product with its previous year's quantity. If this difference is greater than or equal to zero for all the rows, then the product is a constantly increasing in sales. The final query to get the required result is

```
SELECT PRODUCT_NAME

FROM

(
```

```
SELECT P.PRODUCT_NAME,

       S.QUANTITY -

       LEAD(S.QUANTITY,1,0) OVER (

                            PARTITION BY P.PRODUCT_ID

                            ORDER BY S.YEAR DESC

                            ) QUAN_DIFF

FROM   PRODUCTS P,

       SALES S

WHERE  P.PRODUCT_ID = S.PRODUCT_ID

)A

GROUP BY PRODUCT_NAME

HAVING MIN(QUAN_DIFF) >= 0;



PRODUCT_NAME

------------

IPhone
```

2. Write a SQL query to find the products which does not have sales at all?

Solution:

“LG” is the only product which does not have sales at all. This can be achieved in three ways.

Method1: Using left outer join.

```
SELECT P.PRODUCT_NAME

FROM   PRODUCTS P
```

```
       LEFT OUTER JOIN

       SALES S

ON     (P.PRODUCT_ID = S.PRODUCT_ID);

WHERE  S.QUANTITY IS NULL



PRODUCT_NAME

------------

LG
```

Method2: Using the NOT IN operator.

```
SELECT P.PRODUCT_NAME

FROM   PRODUCTS P

WHERE  P.PRODUCT_ID NOT IN

       (SELECT DISTINCT PRODUCT_ID FROM SALES);



PRODUCT_NAME

------------

LG
```

Method3: Using the NOT EXISTS operator.

```
SELECT P.PRODUCT_NAME

FROM   PRODUCTS P
```

```
WHERE  NOT EXISTS

       (SELECT 1 FROM SALES S WHERE S.PRODUCT_ID = P.PRODUCT_ID);



PRODUCT_NAME

------------

LG
```

3. Write a SQL query to find the products whose sales decreased in 2012 compared to 2011?

Solution:

Here Nokia is the only product whose sales decreased in year 2012 when compared with the sales in the year 2011. The SQL query to get the required output is

```
SELECT P.PRODUCT_NAME

FROM   PRODUCTS P,

       SALES S_2012,

       SALES S_2011

WHERE  P.PRODUCT_ID = S_2012.PRODUCT_ID

AND    S_2012.YEAR = 2012

AND    S_2011.YEAR = 2011

AND    S_2012.PRODUCT_ID = S_2011.PRODUCT_ID

AND    S_2012.QUANTITY < S_2011.QUANTITY;



PRODUCT_NAME

------------
```

```
Nokia
```

4. Write a query to select the top product sold in each year?

Solution:

Nokia is the top product sold in the year 2010. Similarly, Samsung in 2011 and IPhone, Samsung in 2012. The query for this is

```
SELECT PRODUCT_NAME,

       YEAR

FROM

(

SELECT P.PRODUCT_NAME,

       S.YEAR,

       RANK() OVER (

              PARTITION BY S.YEAR

              ORDER BY S.QUANTITY DESC

              ) RNK

FROM   PRODUCTS P,

       SALES S

WHERE  P.PRODUCT_ID = S.PRODUCT_ID

) A

WHERE RNK = 1;



PRODUCT_NAME YEAR

--------------------
```

```
Nokia        2010

Samsung      2011

IPhone       2012

Samsung      2012
```

5. Write a query to find the total sales of each product.?

Solution:

This is a simple query. You just need to group by the data on PRODUCT_NAME and then find the sum of sales.

```
SELECT P.PRODUCT_NAME,

       NVL( SUM( S.QUANTITY*S.PRICE ), 0) TOTAL_SALES

FROM   PRODUCTS P

       LEFT OUTER JOIN

       SALES S

ON     (P.PRODUCT_ID = S.PRODUCT_ID)

GROUP BY P.PRODUCT_NAME;



PRODUCT_NAME TOTAL_SALES

---------------------------

LG            0

IPhone        405000

Samsung       406000

Nokia         245000
```

SQL Queries Interview Questions - Oracle Part 2

This is continuation to my previous post, SQL Queries Interview Questions - Oracle Part 1, Where i have used PRODUCTS and SALES tables as an example. Here also i am using the same tables. So, just take a look at the tables by going through that link and it will be easy for you to understand the questions mentioned here.

Solve the below examples by writing SQL queries.

1. Write a query to find the products whose quantity sold in a year should be greater than the average quantity sold across all the years?

Solution:

This can be solved with the help of correlated query. The SQL query for this is

```
SELECT P.PRODUCT_NAME,

       S.YEAR,

       S.QUANTITY

FROM   PRODUCTS P,

       SALES S

WHERE  P.PRODUCT_ID = S.PRODUCT_ID

AND    S.QUANTITY >

       (SELECT AVG(QUANTITY)

       FROM SALES S1

       WHERE S1.PRODUCT_ID = S.PRODUCT_ID

       );



PRODUCT_NAME YEAR QUANTITY

-------------------------

Nokia        2010    25

IPhone       2012    20
```

```
Samsung      2012    20

Samsung      2010    20
```

2. Write a query to compare the products sales of "IPhone" and "Samsung" in each year? The output should look like as

```
YEAR IPHONE_QUANT SAM_QUANT IPHONE_PRICE SAM_PRICE

--------------------------------------------------

2010   10           20       9000         7000

2011   15           18       9000         7000

2012   20           20       9000         7000
```

Solution:

By using self-join SQL query we can get the required result. The required SQL query is

```
SELECT S_I.YEAR,

       S_I.QUANTITY IPHONE_QUANT,

       S_S.QUANTITY SAM_QUANT,

       S_I.PRICE    IPHONE_PRICE,

       S_S.PRICE    SAM_PRICE

FROM   PRODUCTS P_I,

       SALES S_I,

       PRODUCTS P_S,

       SALES S_S

WHERE  P_I.PRODUCT_ID = S_I.PRODUCT_ID
```

```
AND    P_S.PRODUCT_ID = S_S.PRODUCT_ID

AND    P_I.PRODUCT_NAME = 'IPhone'

AND    P_S.PRODUCT_NAME = 'Samsung'

AND    S_I.YEAR = S_S.YEAR
```

3. Write a query to find the ratios of the sales of a product?

Solution:

The ratio of a product is calculated as the total sales price in a particular year divide by the total sales price across all years. Oracle provides RATIO_TO_REPORT analytical function for finding the ratios. The SQL query is

```
SELECT P.PRODUCT_NAME,

       S.YEAR,

       RATIO_TO_REPORT(S.QUANTITY*S.PRICE)

         OVER(PARTITION BY P.PRODUCT_NAME ) SALES_RATIO

FROM   PRODUCTS P,

       SALES S

WHERE (P.PRODUCT_ID = S.PRODUCT_ID);



PRODUCT_NAME YEAR      RATIO

----------------------------

IPhone       2011   0.333333333

IPhone       2012   0.444444444

IPhone       2010   0.222222222

Nokia        2012   0.163265306

Nokia        2011   0.326530612
```

```
Nokia        2010   0.510204082

Samsung      2010   0.344827586

Samsung      2012   0.344827586

Samsung      2011   0.310344828
```

4. In the SALES table quantity of each product is stored in rows for every year. Now write a query to transpose the quantity for each product and display it in columns? The output should look like as

```
PRODUCT_NAME QUAN_2010 QUAN_2011 QUAN_2012

-----------------------------------------

IPhone       10        15        20

Samsung      20        18        20

Nokia        25        16        8
```

Solution:

Oracle 11g provides a pivot function to transpose the row data into column data. The SQL query for this is

```
SELECT * FROM

(

SELECT P.PRODUCT_NAME,

       S.QUANTITY,

       S.YEAR

FROM   PRODUCTS P,

       SALES S

WHERE (P.PRODUCT_ID = S.PRODUCT_ID)
```

```
)A

PIVOT ( MAX(QUANTITY) AS QUAN FOR (YEAR) IN (2010,2011,2012));
```

If you are not running oracle 11g database, then use the below query for transposing the row data into column data.

```
SELECT P.PRODUCT_NAME,

       MAX(DECODE(S.YEAR,2010, S.QUANTITY)) QUAN_2010,

       MAX(DECODE(S.YEAR,2011, S.QUANTITY)) QUAN_2011,

       MAX(DECODE(S.YEAR,2012, S.QUANTITY)) QUAN_2012

FROM   PRODUCTS P,

       SALES S

WHERE (P.PRODUCT_ID = S.PRODUCT_ID)

GROUP BY P.PRODUCT_NAME;
```

5. Write a query to find the number of products sold in each year?

Solution:

To get this result we have to group by on year and the find the count. The SQL query for this question is

```
SELECT YEAR,

       COUNT(1) NUM_PRODUCTS

FROM   SALES

GROUP BY YEAR;



YEAR  NUM_PRODUCTS
```

```
------------------

2010      3

2011      3

2012      3
```

SQL Queries Interview Questions - Oracle Part 3

Here I am providing Oracle SQL Query Interview Questions. If you find any bugs in the queries, Please do comment. So, that i will rectify them.

1. Write a query to generate sequence numbers from 1 to the specified number N?

Solution:

```
SELECT LEVEL FROM DUAL CONNECT BY LEVEL<=&N;
```

2. Write a query to display only friday dates from Jan, 2000 to till now?

Solution:

```
SELECT  C_DATE,

        TO_CHAR(C_DATE,'DY')

FROM

(

  SELECT TO_DATE('01-JAN-2000','DD-MON-YYYY')+LEVEL-1 C_DATE

  FROM   DUAL

  CONNECT BY LEVEL <=

        (SYSDATE - TO_DATE('01-JAN-2000','DD-MON-YYYY')+1)

)

WHERE TO_CHAR(C_DATE,'DY') = 'FRI';
```

3. Write a query to duplicate each row based on the value in the repeat column? The input table data looks like as below

```
Products, Repeat

----------------

A,          3

B,          5

C,          2
```

Now in the output data, the product A should be repeated 3 times, B should be repeated 5 times and C should be repeated 2 times. The output will look like as below

```
Products, Repeat

----------------

A,         3

A,         3

A,         3

B,         5

B,         5

B,         5

B,         5

B,         5

C,         2

C,         2
```

Solution:

```
SELECT PRODUCTS,

       REPEAT

FROM   T,

      ( SELECT LEVEL L FROM DUAL

        CONNECT BY LEVEL <= (SELECT MAX(REPEAT) FROM T)

      ) A

WHERE T.REPEAT >= A.L

ORDER BY T.PRODUCTS;
```

4. Write a query to display each letter of the word "SMILE" in a separate row?

```
S

M

I

L

E
```

Solution:

```
SELECT SUBSTR('SMILE',LEVEL,1) A

FROM   DUAL

CONNECT BY LEVEL <=LENGTH('SMILE');
```

5. Convert the string "SMILE" to Ascii values? The output should look like as 83,77,73,76,69. Where 83 is the ascii value of S and so on.
The ASCII function will give ascii value for only one character. If you pass a string to the ascii function, it will give the ascii value of first letter in the string. Here i am providing two solutions to get the ascii values of string.

Solution1:

```
SELECT SUBSTR(DUMP('SMILE'),15)

FROM DUAL;
```

Solution2:

```
SELECT WM_CONCAT(A)

FROM

(

SELECT ASCII(SUBSTR('SMILE',LEVEL,1)) A

FROM   DUAL

CONNECT BY LEVEL <=LENGTH('SMILE')

);
```

SQL Queries Interview Questions - Oracle Part 4

1. Consider the following friends table as the source

```
Name, Friend_Name

-----------------

sam,   ram

sam,   vamsi
```

```
vamsi, ram

vamsi, jhon

ram,   vijay

ram,   anand
```

Here ram and vamsi are friends of sam; ram and jhon are friends of vamsi and so on. Now write a query to find friends of friends of sam. For sam; ram,jhon,vijay and anand arefriends of friends. The output should look as

```
Name, Friend_of_Firend

----------------------

sam,    ram

sam,    jhon

sam,    vijay

sam,    anand
```

Solution:

```
SELECT  f1.name,

        f2.friend_name as friend_of_friend

FROM    friends f1,

        friends f2

WHERE   f1.name = 'sam'
```

```
AND     f1.friend_name = f2.name;
```

2. This is an extension to the problem 1. In the output, you can see ram is displayed asfriends of friends. This is because, ram is mutual friend of sam and vamsi. Now extend the above query to exclude mutual friends. The outuput should look as

```
Name, Friend_of_Friend

----------------------

sam,    jhon

sam,    vijay

sam,    anand
```

Solution:

```
SELECT  f1.name,

        f2.friend_name as friend_of_friend

FROM    friends f1,

        friends f2

WHERE   f1.name = 'sam'

AND     f1.friend_name = f2.name

AND     NOT EXISTS

        (SELECT 1 FROM friends f3

         WHERE f3.name = f1.name
```

```
        AND   f3.friend_name = f2.friend_name);
```

3. Write a query to get the top 5 products based on the quantity sold without using the row_number analytical function? The source data looks as

```
Products, quantity_sold, year

----------------------------

A,         200,          2009

B,         155,          2009

C,         455,          2009

D,         620,          2009

E,         135,          2009

F,         390,          2009

G,         999,          2010

H,         810,          2010

I,         910,          2010

J,         109,          2010

L,         260,          2010

M,         580,          2010
```

Solution:

```
SELECT  products,
```

```
        quantity_sold,

        year

FROM

(

  SELECT  products,

          quantity_sold,

          year,

          rownum r

  from    t

  ORDER BY quantity_sold DESC

)A

WHERE r <= 5;
```

4. This is an extension to the problem 3. Write a query to produce the same output using row_number analytical function?

Solution:

```
SELECT  products,

        quantity_sold,

        year

FROM

(

  SELECT products,
```

```
         quantity_sold,

         year,

         row_number() OVER(

            ORDER BY quantity_sold DESC) r

  from   t

)A

WHERE r <= 5;
```

5. This is an extension to the problem 3. write a query to get the top 5 products in each year based on the quantity sold?

Solution:

```
SELECT  products,

        quantity_sold,

        year

FROM

(

   SELECT products,

          quantity_sold,

          year,

          row_number() OVER(

               PARTITION BY year

               ORDER BY quantity_sold DESC) r
```

```
   from   t

)A

WHERE r <= 5;
```

SQL Query Interview Questions - Part 5

Write SQL queries for the below interview questions:

1. Load the below products table into the target table.

```
CREATE TABLE PRODUCTS

(

       PRODUCT_ID     INTEGER,

       PRODUCT_NAME   VARCHAR2(30)

);



INSERT INTO PRODUCTS VALUES ( 100, 'Nokia');

INSERT INTO PRODUCTS VALUES ( 200, 'IPhone');

INSERT INTO PRODUCTS VALUES ( 300, 'Samsung');

INSERT INTO PRODUCTS VALUES ( 400, 'LG');

INSERT INTO PRODUCTS VALUES ( 500, 'BlackBerry');

INSERT INTO PRODUCTS VALUES ( 600, 'Motorola');

COMMIT;



SELECT * FROM PRODUCTS;



PRODUCT_ID PRODUCT_NAME
```

```
-----------------------

100         Nokia

200         IPhone

300         Samsung

400         LG

500         BlackBerry

600         Motorola
```

The requirements for loading the target table are:

- Select only 2 products randomly.
- Do not select the products which are already loaded in the target table with in the last 30 days.
- Target table should always contain the products loaded in 30 days. It should not contain the products which are loaded prior to 30 days.

Solution:

First we will create a target table. The target table will have an additional column INSERT_DATE to know when a product is loaded into the target table. The target
table structure is

```
CREATE TABLE TGT_PRODUCTS

(

        PRODUCT_ID      INTEGER,

        PRODUCT_NAME    VARCHAR2(30),

        INSERT_DATE     DATE

);
```

The next step is to pick 5 products randomly and then load into target table. While selecting check whether the products are there in the

```
INSERT INTO TGT_PRODUCTS

SELECT  PRODUCT_ID,

        PRODUCT_NAME,

        SYSDATE INSERT_DATE

FROM

(

SELECT  PRODUCT_ID,

 PRODUCT_NAME

FROM PRODUCTS S

WHERE   NOT EXISTS (

           SELECT 1

           FROM   TGT_PRODUCTS T

           WHERE  T.PRODUCT_ID = S.PRODUCT_ID

        )

ORDER BY DBMS_RANDOM.VALUE --Random number generator in oracle.

)A

WHERE ROWNUM <= 2;
```

The last step is to delete the products from the table which are loaded 30 days back.

```
DELETE FROM TGT_PRODUCTS

WHERE  INSERT_DATE < SYSDATE - 30;
```

2. Load the below CONTENTS table into the target table.

```
CREATE TABLE CONTENTS

(

  CONTENT_ID  INTEGER,

  CONTENT_TYPE VARCHAR2(30)

);



INSERT INTO CONTENTS VALUES (1,'MOVIE');

INSERT INTO CONTENTS VALUES (2,'MOVIE');

INSERT INTO CONTENTS VALUES (3,'AUDIO');

INSERT INTO CONTENTS VALUES (4,'AUDIO');

INSERT INTO CONTENTS VALUES (5,'MAGAZINE');

INSERT INTO CONTENTS VALUES (6,'MAGAZINE');

COMMIT;



SELECT * FROM CONTENTS;



CONTENT_ID CONTENT_TYPE

-----------------------

1          MOVIE

2          MOVIE

3          AUDIO

4          AUDIO

5          MAGAZINE

6          MAGAZINE
```

The requirements to load the target table are:

- Load only one content type at a time into the target table.
- The target table should always contain only one contain type.
- The loading of content types should follow round-robin style. First MOVIE, second AUDIO, Third MAGAZINE and again fourth Movie.

Solution:

First we will create a lookup table where we mention the priorities for the content types. The lookup table “Create Statement” and data is shown below.

```
CREATE TABLE CONTENTS_LKP

(

  CONTENT_TYPE VARCHAR2(30),

  PRIORITY     INTEGER,

  LOAD_FLAG  INTEGER

);



INSERT INTO CONTENTS_LKP VALUES('MOVIE',1,1);

INSERT INTO CONTENTS_LKP VALUES('AUDIO',2,0);

INSERT INTO CONTENTS_LKP VALUES('MAGAZINE',3,0);

COMMIT;



SELECT * FROM CONTENTS_LKP;



CONTENT_TYPE PRIORITY LOAD_FLAG

---------------------------------

MOVIE         1          1
```

```
AUDIO          2           0

MAGAZINE       3           0
```

Here if LOAD_FLAG is 1, then it indicates which content type needs to be loaded into the target table. Only one content type will have LOAD_FLAG as 1. The other content types will have LOAD_FLAG as 0. The target table structure is same as the source table structure.

The second step is to truncate the target table before loading the data

```
TRUNCATE TABLE TGT_CONTENTS;
```

The third step is to choose the appropriate content type from the lookup table to load the source data into the target table.

```
INSERT INTO TGT_CONTENTS

SELECT  CONTENT_ID,

 CONTENT_TYPE

FROM CONTENTS

WHERE CONTENT_TYPE = (SELECT CONTENT_TYPE FROM CONTENTS_LKP WHERE
LOAD_FLAG=1);
```

The last step is to update the LOAD_FLAG of the Lookup table.

```
UPDATE CONTENTS_LKP

SET LOAD_FLAG = 0

WHERE LOAD_FLAG = 1;



UPDATE CONTENTS_LKP

SET LOAD_FLAG = 1
```

```
WHERE PRIORITY = (

SELECT DECODE( PRIORITY,(SELECT MAX(PRIORITY) FROM CONTENTS_LKP) ,1 ,
PRIORITY+1)

FROM   CONTENTS_LKP

WHERE  CONTENT_TYPE = (SELECT DISTINCT CONTENT_TYPE FROM TGT_CONTENTS)

);
```